Assignment 1A — Building & Fine-Tuning a Domain-Specific LLM

End-to-End Pipeline: Continual Pre-Training to QLoRA Instruction Fine-Tuning

Component	Part	Marks
Continual Pre-Training (CPT)	Part A	10
QLoRA Instruction Fine-Tuning	Part B	5
TOTAL		**15**

Completing this end-to-end pipeline — from raw PDFs to a continually pre-trained, instruction-fine-tuned domain LLM — gives students direct, hands-on experience with the exact workflow used in production AI teams.

Domain & Model Selection

Select one domain from the table below. The same domain corpus is used across both Part A and Part B. Choose the model based on the GPU available. Use the tokenizer of your chosen model throughout — do not mix tokenizers across steps.

T4 GPU (16 GB VRAM) — available on free Colab. A100 GPU/L40S — BITS remote lab
Both model choices use the same assignment pipeline; only scale differs.
The following table is for your reference. You are free to choose any domain and any model.

Use Case / Domain	T4 GPU — Choice 1	T4 GPU — Choice 2	A100 GPU — Choice 1	A100 GPU — Choice 2
Medical & Clinical Literature	**BioGPT-Large** (347 M) `microsoft/biogpt-large`	**GPT-2-Medium** (345 M) `openai-community/gpt2-medium`	**BioMedLM** (2.7 B) `stanford-crfm/BioMedLM`	**Mistral-7B-v0.1** (7 B) `mistralai/Mistral-7B-v0.1`
Legal Documents & Contracts	**SmolLM2-360M** (360 M) `HuggingFaceTB/SmolLM2-360M`	**GPT-2-Medium** (345 M) `openai-community/gpt2-medium`	**Mistral-7B-v0.1** (7 B) `mistralai/Mistral-7B-v0.1`	**Qwen2.5-7B** (7 B) `Qwen/Qwen2.5-7B`
Financial Reports & Filings	**SmolLM2-360M** (360 M) `HuggingFaceTB/SmolLM2-360M`	**TinyLlama-1.1B** (1.1 B) `TinyLlama/TinyLlama-1.1B-…`	**Qwen2.5-7B** (7 B) `Qwen/Qwen2.5-7B`	**Mistral-7B-v0.1** (7 B) `mistralai/Mistral-7B-v0.1`
Manufacturing & Technical Standards	**TinyLlama-1.1B** (1.1 B) `TinyLlama/TinyLlama-1.1B-…`	**SmolLM2-360M** (360 M) `HuggingFaceTB/SmolLM2-360M`	**Mistral-7B-v0.1** (7 B) `mistralai/Mistral-7B-v0.1`	**LLaMA-3-8B** (8 B) `meta-llama/Meta-Llama-3-8B`
HR Policy & Corporate Documents	**SmolLM2-1.7B** (1.7 B) `HuggingFaceTB/SmolLM2-1.7B`	**TinyLlama-1.1B** (1.1 B) `TinyLlama/TinyLlama-1.1B-…`	**Gemma-7B** (7 B) `google/gemma-7b`	**LLaMA-3-8B** (8 B) `meta-llama/Meta-Llama-3-8B`
Scientific Research Papers	**SmolLM2-360M** (360 M) `HuggingFaceTB/SmolLM2-360M`	**GPT-2-Large** (774 M) `openai-community/gpt2-large`	**LLaMA-3-8B** (8 B) `meta-llama/Meta-Llama-3-8B`	**Qwen2.5-7B** (7 B) `Qwen/Qwen2.5-7B`

Always load the tokenizer paired with your chosen model using AutoTokenizer.from_pretrained('<model-id>'). Do not swap tokenizers between models — vocabulary alignment breaks CPT.

PART A — Continual Pre-Training (CPT) for Domain Adaptation [10 Marks]

Detailed inferences and justification for all the tasks are mandatory.

Step 1 — Data Collection, Extraction & Cleaning [2 Marks]

Collect domain-specific PDF documents. You can increase the size of your data if high performance GPU is available. Write a Python script to extract text page-by-page from all PDFs and save each document as a .txt file. Use any standard PDF extraction library.

Apply the following step cleaning pipeline to the raw extracted text: (not confined to the following)

- **Length filter**
- **Deduplication**
- **Language filter —** retain only English-language documents

Report document counts before and after each cleaning step. Identify which step had the greatest impact on corpus size.

Step 2 — Tokenization & Packed Dataset [2 Marks]

Load the tokenizer of your chosen model. Do not train a custom tokenizer — the model's pretrained tokenizer must be reused to preserve vocabulary alignment during CPT.

- **BOS / EOS tokens —** wrap each document's token IDs with a Beginning-of-Sequence token at the start and an End-of-Sequence token at the end, marking document boundaries within packed sequences
- **Sequence packing —** concatenate all token IDs from all documents into one flat stream, then slice into fixed-length chunks matching the model's context window. This avoids padding and achieves full GPU utilisation
- **Save as Parquet**

  Report total token count, average document length in tokens, and total number of packed sequences produced

Step 3 — Model Loading & Architecture Inspection [2 Marks]

Load your chosen model using AutoModelForCausalLM.from_pretrained(). For T4 GPU, load in bfloat16 precision with gradient checkpointing enabled to reduce memory. For A100, full precision or bfloat16 both work.

- **Parameter count —** report total trainable parameters
- **Architecture audit —** document number of decoder layers, attention heads, hidden size, and head dimension from the model config
- **lm_head check —** confirm the output dimension of the prediction head equals the vocabulary size
- **Baseline inference —** run the pretrained base model (before any CPT) on 3 domain-specific prompts and save the generated text as a comparison baseline for Step 4

Step 4 — CPT Training Loop & Loss Analysis [2 Marks]

Wrap the packed dataset in a PyTorch Dataset class and configure training hyperparameters.

- **Loss callback —** implement a custom TrainerCallback to capture training loss at every logging step
- **Loss curve —** plot loss vs. training step; verify loss decreases from the pretrained starting point (~2–4) and identify where it plateaus
- **Checkpoint —** save the final CPT model and tokenizer to persistent storage for use in Step 5 and Part B

Tip: A pretrained model starts CPT at loss ≈ 2–4. If your starting loss is ≈ 10.8, the model initialised randomly — investigate your loading step. If loss diverges or spikes, reduce the learning rate or increase warmup steps.

Step 5 — Evaluation: Perplexity & Catastrophic Forgetting [2 Marks]

Perplexity (PPL) measures how well a language model predicts a held-out text sample. Formally, it is the exponentiated average negative log-likelihood per token:

$$\mathbf{PPL = \exp\left(-(1/N) \times \Sigma \log P(t_i \mid t_1 \ldots t_{i-1}) \right)}$$

where N is the total number of tokens and $P(t_i \mid \ldots)$ is the model's predicted probability of the correct next token.

Intuitively: a model that always predicts the next token correctly achieves PPL → 1 (perfect). A randomly initialised model over a vocabulary of ~49K tokens scores PPL ≈ 49,000. A well-pretrained general model scores PPL ≈ 15–30 on standard benchmarks.

After CPT on your domain, PPL on domain text should decrease — confirming that the model has learned the statistical patterns, vocabulary, and sentence structure of your specific domain.

Key rule: Lower domain PPL after CPT = successful domain adaptation.

5A — Domain Perplexity

Reserve 10% of your domain corpus as a held-out evaluation set (never seen during CPT training). Compute perplexity on this split for both the base model and the CPT model by running a standard cross-entropy loss evaluation loop (no gradients, no training).

- **Base model PPL —** perplexity of the original pretrained model on your domain evaluation set
- **CPT model PPL —** perplexity of the CPT model on the same evaluation set
- **PPL reduction —** report the percentage drop. A successful CPT run typically reduces domain perplexity by 10–40%

5B — Catastrophic Forgetting Check

CPT can cause the model to lose general world knowledge it gained during original pretraining. This is called catastrophic forgetting. Test both models on 3 general-domain prompts completely unrelated to your domain — for example:

- "The capital of France is…"
- "Water boils at…"
- "The speed of light is approximately…"

Present outputs side-by-side in a comparison table with a verdict column (Retained / Degraded).

> *Tip: If general outputs degrade significantly, your learning rate was too high or training ran too many steps. Reducing the learning rate by 10× or halving max_steps usually mitigates forgetting while preserving domain gains.*

PART B — Instruction Fine-Tuning with QLoRA [5 Marks]

This part extends the CPT model from Part A into an instruction-following assistant for your domain. Use the same domain corpus and the CPT checkpoint saved in Step 4. You may use the same model or switch to a larger model if A100 is available.

Part B1 — Instruction Dataset Creation [2 Marks]

Transform your cleaned domain corpus into a structured instruction dataset suitable for supervised fine-tuning.

- **Source —** use the cleaned .txt files from Part A, Step 1. Generate suitable size for instructional dataset appropriate for your chosen model.
- **Format —**JSONL format; each entry must contain instruction (a user question) and response (an answer derived directly from your domain text)
- **Method —** use manual/heuristic generation or synthetic generation via an external LLM. If using an LLM, include the exact prompt template you used
- **Split —** divide into 80% training and 20% evaluation sets; report the counts for both

Synthetic generation tip: 'Read the text below and generate 10 instruction-response pairs in JSON format based ONLY on this text. Each entry must have instruction and response keys.'

Part B2 — QLoRA Fine-Tuning with One Adapter Configurations [2 Marks]

Apply QLoRA using the transformers, peft, and bitsandbytes (4-bit quantization) libraries. Train any one of the following adapter on your instruction training set using SFTTrainer formatted with the model's chat template.

Adapter	LoRA Rank (r)	LoRA Alpha	Target Modules	Expected Effect
Adapter A (Low Capacity)	8	16	q_proj, v_proj	Faster; may underfit complex domain queries
Adapter B (Balanced)	16	32	q_proj, v_proj	Good quality / cost trade-off
Adapter C (High Capacity)	32	32	q_proj, v_proj, o_proj	Best quality; slower; higher VRAM usage

Part B3 — Evaluation Analysis [1 Mark]

Run your trained adapter on the same 3 domain prompts. Present your observations.

Submission Deliverables

File	Contents
`Python Notebook with Output`	**Includes** • Steps 1–5: Data pipeline → CPT training → perplexity + catastrophic-forgetting evaluation • B1–B3: Instruction dataset creation → QLoRA adapter training → comparative evaluation Detailed inferences and observations are mandatory.
`HTML file of .ipynb with Output`	Exported HTML version of the notebook with outputs.
`instruction_dataset.jsonl`	Final instruction–response pairs (JSONL format).
`domain_corpus/*.txt`	Cleaned domain text files generated in Step 1.

Optional Extension (No Marks)

Build a CLI or notebook chat loop that routes domain queries to different adapters using keyword rules or intent-based routing. Explore task-specific adapters (summarization, Q&A, sentiment) and demonstrate multi-adapter switching with model.set_adapter().